REINFORCE Explained

The First Policy Gradient Algorithm

1 Goal of This Note

REINFORCE is one of the simplest policy gradient algorithms. It trains a policy directly by making rewarded actions more likely.

The essential idea is:

sample actions → observe return → increase probability of actions that produced high return

This is the algorithm behind the loss from Lesson 1:

$$\mathcal{L}(\theta) = -\sum_t \log \pi_\theta(a_t \mid s_t) G_t.$$

2 The Learning Problem

The policy is a probability distribution:

$$\pi_\theta(a \mid s).$$

The parameters θ are the weights of the neural network. Our goal is to maximize expected return:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)].$$

Here τ is a trajectory:

$$\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \ldots).$$

The problem is that the reward is usually not differentiable with respect to the neural network weights. The environment is outside the neural network. We cannot backpropagate through the real world, a simulator, or a game engine in the usual supervised-learning way.

3 The Trick

Even if the reward itself is not differentiable, the probability of the sampled actions is differentiable.

The policy created the data. So we can ask:

Which actions did the policy choose, and how much return followed?

If an action was followed by high return, make that action more likely in similar states. If an action was followed by low return, make it less likely.

4 The Score Function Identity

The key mathematical identity is:

$$\nabla_\theta p_\theta(x) = p_\theta(x) \nabla_\theta \log p_\theta(x).$$

This follows from:

$$\nabla_\theta \log p_\theta(x) = \frac{\nabla_\theta p_\theta(x)}{p_\theta(x)}.$$

Multiplying both sides by $p_\theta(x)$ gives the identity.
This identity lets us move gradients onto log-probabilities.

5 Policy Gradient Formula

The objective is:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}[G(\tau)].$$

Using the score function identity:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[G(\tau) \nabla_\theta \log P_\theta(\tau) \right].$$

The trajectory probability contains the policy terms:

$$P_\theta(\tau) = p(s_0) \prod_t \pi_\theta(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t).$$

The environment transition probabilities do not depend on θ. So the gradient only sees the policy:

$$\nabla_\theta \log P_\theta(\tau) = \sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t).$$

Therefore:

$$\boxed{\nabla_\theta J(\theta) = \mathbb{E}\left[\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t \right].}$$

6 From Maximization to a Loss

PyTorch minimizes losses. But REINFORCE wants to maximize expected return.

So we minimize the negative objective:

$$\boxed{\mathcal{L}(\theta) = -\sum_t \log \pi_\theta(a_t \mid s_t) G_t.}$$

If G_t is high, minimizing this loss increases $\log \pi_\theta(a_t \mid s_t)$. That means the action becomes more likely.

If G_t is low or negative, minimizing the loss decreases that log-probability. That action becomes less likely.

7 One Episode REINFORCE

The simplest version performs one gradient update after one episode.

```
log_probs = []
rewards = []

state, _ = env.reset()
done = False

while not done:
    state = torch.tensor(state, dtype=torch.float32)
    logits = policy(state)
    dist = torch.distributions.Categorical(logits=logits)

    action = dist.sample()
    next_state, reward, terminated, truncated, _ = env.step(action.item())
    done = terminated or truncated

    log_probs.append(dist.log_prob(action))
    rewards.append(reward)
    state = next_state
```

After the episode:

```
returns = discounted_returns(rewards, gamma=0.99)
returns = (returns - returns.mean()) / (returns.std() + 1e-8)

loss = 0
for log_prob, G in zip(log_probs, returns):
    loss += -log_prob * G

optimizer.zero_grad()
loss.backward()
optimizer.step()
```

8 Why We Need Full Episodes

In REINFORCE, the return G_t is computed after observing future rewards.

For a time step t:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \cdots .$$

This means we usually wait until the episode ends before updating. Only then do we know the full future return for each action.

9 Credit Assignment

Credit assignment means deciding which actions deserve credit for future reward.

If an episode lasted 500 steps, many actions contributed. REINFORCE gives each action a return:

$$\log \pi(a_t \mid s_t) G_t.$$

Actions earlier in the episode get credit for the future reward that followed them. Actions later in the episode get credit for a shorter future.

10 Variance

REINFORCE is simple, but noisy. Two similar episodes can produce different returns because actions are sampled randomly.

The gradient estimate:

$$\sum_t \nabla_\theta \log \pi_\theta(a_t \mid s_t) G_t$$

can have high variance. High variance means learning may be unstable.

11 Baseline

A baseline reduces variance. The most common baseline is the value function:

$$b(s_t) = V(s_t).$$

Instead of using return directly, we use advantage:

$$A_t = G_t - V(s_t).$$

The loss becomes:

$$\mathcal{L}(\theta) = -\sum_t \log \pi_\theta(a_t \mid s_t) A_t.$$

The intuition is:

> Do not reward an action just because the episode was good.
> Reward it if it was better than expected.

12 Tiny Actor-Critic Step

If we add a value network, we can train both a policy and a baseline.

```
logits = policy(state)
value = critic(state)

dist = Categorical(logits=logits)
action = dist.sample()
log_prob = dist.log_prob(action)

advantage = return_t - value.detach()
policy_loss = -log_prob * advantage
value_loss = (value - return_t).pow(2)
```

```
loss = policy_loss + 0.5 * value_loss
```

This is the beginning of actor-critic methods. The actor chooses actions. The critic estimates values.

13 Entropy Bonus

Early in training, the policy may become too confident too quickly. An entropy bonus encourages exploration:

$$\mathcal{L} = \mathcal{L}_{\text{policy}} - \beta H(\pi).$$

```
entropy = dist.entropy()
loss = policy_loss - 0.01 * entropy
```

This says: minimize policy loss, but prefer policies that still explore.

14 Complete Minimal Training Loop

```
for episode in range(num_episodes):
    log_probs, rewards = collect_episode(env, policy)
    returns = discounted_returns(rewards, gamma=0.99)
    returns = normalize(returns)

    loss = sum(-lp * G for lp, G in zip(log_probs, returns))

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

This is REINFORCE in its most compact form.

15 Connection to Supervised Learning

In supervised learning, the dataset tells us the correct action:

$$y = \text{correct class}.$$

The loss is:

$$-\log p_\theta(y \mid x).$$

In REINFORCE, the policy samples the action:

$$a_t \sim \pi_\theta(\cdot \mid s_t).$$

The environment tells us how good the outcome was:

$$G_t.$$

The loss is:

$$-G_t \log \pi_\theta(a_t \mid s_t).$$

So the target action is not given by a teacher. It is generated by the policy and judged by reward.

16 Strengths and Weaknesses

Strength	Weakness
Very simple	High variance
Works with discrete or continuous actions	Often sample inefficient
Directly optimizes the policy	Needs exploration
Does not require a Q-table	Can learn slowly

17 Minimal Mental Model

REINFORCE is:

> log loss where the label is the sampled action and the weight is future reward.

The whole algorithm can be remembered as:

> If an action led to better return, make it more probable next time.